

Laboratorium 8

Symulator Dostaw i Logistyki

Programowanie Obiektowe w C++

1. Cel laboratorium

Twoim zadaniem jest stworzenie systemu obsługującego przesyłki w firmie kurierskiej. Projekt ma ćwiczyć dziedziczenie, polimorfizm, interfejsy, wzorzec strategii oraz elementy programowania funkcyjnego (lambda i algorytmy STL).

2. Etap 1: Klasy bazowe i typy przesyłek

Stwórz abstrakcyjną klasę `Shipment`.

Pola:

- `std::string id`,
- `double weight`,
- `std::string destinationCity`,
- `bool delivered`.

Metody:

- konstruktor ustawiający pola (`delivered = false`),
- gettery,
- `void markDelivered()`,
- `virtual double shippingCost() const = 0`;
- `virtual std::string typeName() const = 0`;

Dodaj klasy pochodne:

1. `StandardShipment` – koszt: $8 + 2.5 * weight$,
2. `ExpressShipment` – koszt: $15 + 4.0 * weight$,
3. `FragileShipment` – koszt: $12 + 3.0 * weight + 10$ (opłata za ostrożność).

3. Etap 2: Interfejs trasowania

Zdefiniuj interfejs:

- `class IRoutingStrategy`
- metoda: `virtual std::string chooseRoute(const Shipment& shipment) const = 0`;

Zaimplementuj strategie:

- `CheapRouteStrategy`,
- `FastRouteStrategy`,

- `BalancedRouteStrategy`.

Każda strategia zwraca inny opis trasy (np. przez magazyn regionalny, bezpośrednio, mieszanie).

4. Etap 3: Klasa `DeliveryCenter`

Stwórz klasę `DeliveryCenter`.

Pola:

- `std::vector<std::unique_ptr<Shipment>> shipments`,
- `std::unique_ptr<IRoutingStrategy> strategy`,
- `double revenue`.

Metody:

- `void addShipment(std::unique_ptr<Shipment> s)`,
- `void setStrategy(std::unique_ptr<IRoutingStrategy> s)`,
- `void processAll()`,
- `double getRevenue() const`,
- `void printPending() const`,
- `void printDelivered() const`.

Logika:

- `processAll()` dla każdej niedostarczonej przesyłki:
 - wybiera trasę przez aktywną strategię,
 - wypisuje komunikat o obsłudze,
 - oznacza przesyłkę jako dostarczona,
 - dolicza koszt do `revenue`.

5. Etap 4: Wyjątki

Dodaj wyjątki:

- `class LogisticsException : public std::runtime_error`,
- `class InvalidShipmentException : public LogisticsException`,
- `class MissingStrategyException : public LogisticsException`.

Wymagania:

- `addShipment(...)` rzuca `InvalidShipmentException`, gdy `weight <= 0`,
- `processAll()` rzuca `MissingStrategyException`, gdy strategia nie jest ustawiona.

6. Etap 5: Algorytmy STL

Dodaj metody:

- `int countByCity(const std::string& city) const`,
- `double averageWeightDelivered() const`,
- `std::vector<std::string> idsAboveWeight(double threshold) const`.

Wymagania:

- `countByCity` używa `std::count_if`,

- `averageWeightDelivered` używa `std::accumulate`,
- `idsAboveWeight` używa `std::copy_if` i transformacji do samych ID.

7. Etap 6 (dla chetnych): Raport dzienny

Dodaj generator raportu:

- `std::string buildDailyReport() const`

Raport powinien zawierać:

- liczbe przesyłek łącznie,
- liczbe dostarczonych,
- przychód,
- średnia wage dostarczonych,
- 3 najcięższe przesyłki (ID i masa).

8. Co oddać

- pliki `.hpp/.cpp`,
- `main.cpp` z testami etapów 1–6,
- `README.md` z opisem architektury (wzorzec strategii + decyzje projektowe).

9. Kryteria oceny

1. poprawna hierarchia klas i polimorfizm,
2. poprawne wykorzystanie strategii i `unique_ptr`,
3. obsługa wyjątków i walidacja danych,
4. poprawne użycie algorytmów STL,
5. czytelność i modularność kodu.

Wskazówka: najpierw uruchom minimalny przepływ (dodanie i przetworzenie jednej przesyłki), dopiero potem dokładaj kolejne etapy.